Progetto di Algoritmi e Strutture Dati

Cammini minimi su griglie 8-connesse con l'algoritmo CamminoMin

Ergys Perdeda — progetto svolto singolarmente

Università degli Studi di Brescia, A.A. 2024/2025

Indice

1 Introduzione **2**
- 1.1 Strumenti . . . 2
- 1.2 Organizzazione del codice . . . 2
- 1.3 Stato di una cella . . . 3

2 Compito Uno: generazione delle griglie **5**
- 2.1 Parametri . . . 5
- 2.2 Algoritmo . . . 5
- 2.3 Griglie immutabili . . . 6
- 2.4 Persistenza in JSON . . . 6

3 Compito Due: contesto, complemento e distanza libera **7**
- 3.1 Distanza libera . . . 7
- 3.2 Calcolo di contesto e complemento . . . 7
- 3.3 Frontiera . . . 9
- 3.4 Complessità . . . 10
- 3.5 Applicazione . . . 10
- 3.6 Verifica . . . 11

4 Compito Tre: l'algoritmo CamminoMin **12**
- 4.1 Idea e pseudocodice di riferimento . . . 12
- 4.2 Implementazione . . . 12
- 4.3 Ostacoli nelle chiamate annidate . . . 14
- 4.4 Sequenza dei landmark . . . 14
- 4.5 Ottimizzazioni . . . 14
- 4.6 Varianti . . . 15
- 4.7 Contatori e riassunto . . . 16

5 Compito Quattro: analisi e sperimentazione **17**
- 5.1 Analisi teorica . . . 17
- 5.2 Piano sperimentale e protocollo . . . 17
- 5.3 Verifica della correttezza . . . 19
- 5.4 Risultati del piano . . . 19
- 5.5 Effetto delle ottimizzazioni . . . 22
- 5.6 Limiti di scala . . . 23

6 Conclusioni **26**
- 6.1 Limitazioni . . . 26
- 6.2 Sviluppi futuri . . . 26

7 Utilizzo del codice **27**
- 7.1 Contenuto della consegna . . . 27
- 7.2 Requisiti . . . 27
- 7.3 Esecuzione . . . 27
- 7.4 Compilazione e test . . . 28
- 7.5 Grafici e relazione . . . 28

A Formato dei file di output **29**
- A.1 `esperimenti.csv` . . . 29
- A.2 `limiti_scala.csv` . . . 29
- A.3 `riassunti.txt` . . . 30
- A.4 Griglie JSON . . . 30

1 Introduzione

La relazione descrive l'implementazione in Java del progetto di Algoritmi e Strutture Dati dell'A.A. 2024/25: il calcolo di cammini minimi su griglie 8-connesse con ostacoli mediante l'algoritmo ricorsivo CamminoMin proposto dalla traccia [1]. Il progetto è stato svolto singolarmente, secondo i requisiti previsti per i gruppi di una persona.

Il lavoro segue i quattro compiti della traccia:

- **Generazione delle griglie** (sezione 2): un generatore di griglie con ostacoli in posizione casuale, riproducibili tramite seed e salvate in JSON.
- **Contesto, complemento e distanza libera** (sezione 3): il calcolo di contesto, complemento e frontiera di una cella in un'unica passata di costo lineare nel numero di celle.
- **Algoritmo CamminoMin** (sezione 4): l'implementazione ricorsiva, con ordinamento della frontiera, potatura e memoizzazione, che restituisce la lunghezza del cammino minimo e la sequenza dei landmark; una variante propaga alle chiamate annidate il limite del cammino migliore già trovato.
- **Sperimentazione** (sezione 5): l'analisi teorica di tempo e spazio, un piano di 67 istanze che confronta quattro varianti dell'algoritmo con verifica di correttezza, e una serie che individua i limiti di scala delle due varianti principali.

Le ultime due sezioni riportano le conclusioni e le istruzioni per usare il codice.

1.1 Strumenti

Il progetto è scritto in Java ed è compilato da Maven con `--release 17`: il bytecode e le API usate sono quelli di Java 17. Build, test e misure sono stati eseguiti con Eclipse Temurin 21.0.6. Le dipendenze esterne sono due: `org.json` per leggere e scrivere le griglie e JUnit 5 per i test. Maven scarica queste dipendenze, esegue i test e produce un JAR eseguibile che include `org.json`: per usare il programma basta quindi Java, mentre Maven serve solo per ricompilarlo. I grafici della sperimentazione sono generati da uno script Python con `pandas` e `matplotlib`.

1.2 Organizzazione del codice

Il codice applicativo si trova in `src/main/java/mio`, diviso in un package per compito (tabella 1). I test sono in `src/test/java/mio` e seguono la stessa suddivisione.

Package	Contenuto
`mio.griglia`	Interfacce della griglia e delle celle, matrice di base (`GrigliaMatrix`) e vista immutabile con ostacoli aggiuntivi (`GrigliaOstacoliView`)
`mio.compito1`	Generatore casuale (`GeneratoreGriglia`) e lettura e scrittura JSON (`LettoreGriglia`)
`mio.compito2`	Stato della cella (`StatoCella`), distanza libera (`DistanzaLibera`), chiusura e frontiera (`CalcoloChiusura`, `GrigliaConOrigine`)
`mio.compito3`	Algoritmo `CamminoMin` e sue varianti (`VarianteCamminoMin`), landmark (`Landmark`) e risultato (`Cammino`)
`mio.compito4`	Esecuzione nei due versi e raccolta delle metriche (`Sperimentazione`, `StatisticheEsecuzione`), scrittura del CSV (`ScritturaFile`) e serie sui limiti di scala (`LimitiScala`)
`mio.Main`	Piano sperimentale e interfaccia a riga di comando per i Compiti Due e Tre

Tabella 1: Package del progetto.

1.3 Stato di una cella

Durante il calcolo della chiusura ogni cella accumula più informazioni: può essere un ostacolo, appartenere al contesto o al complemento, trovarsi su una delle otto linee rette uscenti dall'origine, essere l'origine stessa o una cella di frontiera. Alcune proprietà si escludono: un ostacolo non appartiene mai alla chiusura e, dopo la passata finale del calcolo (sezione 3.2), nessuna cella è insieme nel contesto e nel complemento. Altre si sommano: l'origine appartiene anche al contesto e una cella di frontiera appartiene sempre al contesto o al complemento. Per rappresentare ogni combinazione con un solo valore, lo stato di una cella è un intero in cui ciascun bit corrisponde a una proprietà.

L'enumerazione `StatoCella` associa a ogni proprietà un bit (tabella 2) e due interi: `value`, cioè i bit da accendere per assegnare la proprietà, e `mask`, cioè i bit da controllare per verificarla; per le sei proprietà i due interi coincidono con il bit corrispondente. L'enumerazione offre quattro operazioni, ciascuna ridotta a una sola operazione bit a bit e quindi di costo $O(1)$:

$$\texttt{is}(s) \iff (s \,\&\, \mathit{mask}) \neq 0, \qquad \texttt{addTo}(s) = s \mid \mathit{value},$$
$$\texttt{isNot}(s) \iff (s \,\&\, \mathit{mask}) = 0, \qquad \texttt{removeTo}(s) = s \,\&\, \neg \mathit{mask}.$$

Bit	Nome	Maschera	Significato
0	`CONTESTO`	`00000001`	raggiungibile con un cammino libero di tipo 1
1	`REGINA`	`00000010`	su una delle otto linee rette uscenti da O
2	`ORIGINE`	`00000100`	origine della chiusura
3	`COMPLEMENTO`	`00001000`	raggiungibile solo con un cammino libero di tipo 2
4	`FRONTIERA`	`00010000`	cella di frontiera
5–6	—	—	non usati
7	`OSTACOLO`	`10000000`	cella non attraversabile
—	`CHIUSURA`	`00001001`	contesto oppure complemento

Tabella 2: Bit dello stato di una cella.

`CHIUSURA` non ha bit da assegnare, perché non è una proprietà autonoma, ma la sua maschera contiene i bit di contesto e complemento: un solo `CHIUSURA.is(s)` stabilisce se la cella appartiene alla chiusura. È il controllo del caso base di CamminoMin (sezione 4.2). I landmark non hanno un bit: sono conservati come lista nel risultato di CamminoMin. Un esempio degli stati prodotti dal calcolo della chiusura è nella tabella 4.

Lo stato occupa un `int`, anche se otto bit starebbero in un `byte`: è il tipo restituito da `ICella.stato()` e usato nel formato JSON. Una matrice di `byte` ridurrebbe di quattro volte la memoria di ogni chiusura; la sezione 5.1 mostra però che nei casi critici la memoria è occupata soprattutto dalla cache di CamminoMin, non dalle matrici.

2 Compito Uno: generazione delle griglie

2.1 Parametri

Il generatore `GeneratoreGriglia.genera(W, H, N, seed)` produce una griglia di $W \times H$ celle con esattamente N ostacoli in posizioni casuali. L'utente può configurare:

- le dimensioni W e H, che devono essere positive;
- il numero di ostacoli N, con $0 \le N \le WH$; dalla riga di comando si impone anche $N < WH$, perché servono almeno una cella libera per l'origine e una per la destinazione;
- il *seed* del generatore pseudo-casuale.

La densità degli ostacoli $N/(WH)$ è il parametro che più influenza il costo di CamminoMin (sezione 5.6). Il seed rende ogni griglia riproducibile: la stessa quaterna (W, H, N, seed) produce sempre la stessa griglia, perché `java.util.Random` è un generatore deterministico a partire dal seed [3].

2.2 Algoritmo

Le celle vengono estratte uniformemente; una matrice booleana evita di scegliere due volte la stessa cella. Le celle estratte sono raccolte in una lista, che viene aggiunta alla griglia con un'unica operazione quando sono stati scelti esattamente N ostacoli (algoritmo 1).

Algoritmo 1 Generazione di una griglia con ostacoli casuali

Ingresso: dimensioni $W, H > 0$, numero di ostacoli $0 \le N \le WH$, *seed*
Uscita: griglia $W \times H$ con esattamente N ostacoli

```
1:  procedure GeneraGriglia(W, H, N, seed)
2:      g ← griglia W × H con tutte le celle attraversabili
3:      occupata ← matrice booleana H × W inizializzata a false
4:      rng ← Random(seed)
5:      L ← ⟨⟩                                   ▷ celle scelte come ostacoli
6:      while |L| < N do
7:          x ← rng.Intero(W);    y ← rng.Intero(H)
8:          if not occupata[y][x] then
9:              occupata[y][x] ← true
10:             aggiungi (x, y) a L
11:     return g.AggiungiOstacoli(L)
```

Costo. Quando sono già stati collocati i ostacoli, un'estrazione trova una cella libera con probabilità $(WH - i)/(WH)$. Il numero atteso di estrazioni è quindi

$$\sum_{i=0}^{N-1} \frac{WH}{WH - i},$$

che resta al più $2N$ finché $N \le WH/2$. L'inizializzazione della matrice e l'unica vista finale (sezione 2.3) costano $O(WH)$, quindi il costo atteso è $O(WH)$ per $N \le WH/2$; avvicinandosi a $N = WH$ le estrazioni attese crescono fino a $\Theta(WH \log WH)$. Gli ostacoli sono aggiunti con un'unica vista: aggiungerli uno alla volta copierebbe la mappa di bit a ogni passo, con costo $O(N \cdot WH)$.

2.3 Griglie immutabili

Aggiungere un ostacolo non modifica la griglia esistente: il metodo `addObstacle` restituisce una nuova `GrigliaOstacoliView`, formata dalla griglia di base e da un `BitSet` [4] con tutti gli ostacoli, quelli della base e quelli aggiunti. Se la griglia di partenza è già una vista, il suo `BitSet` viene copiato ed esteso, così le viste non formano catene: verificare se una cella è un ostacolo costa $O(1)$ e non alloca oggetti. Lo stesso `BitSet`, mai modificato dopo la costruzione, è usato come chiave della cache di CamminoMin (sezione 4.5).

Questa scelta serve soprattutto al Compito Tre. Ogni chiamata ricorsiva di CamminoMin lavora su una griglia in cui la chiusura della chiamata precedente è diventata un insieme di ostacoli; i rami ricorsivi fratelli devono però partire dalla stessa griglia. Con viste immutabili ogni ramo riceve la propria griglia e nessun ramo vede gli ostacoli aggiunti da un altro.

2.4 Persistenza in JSON

`LettoreGriglia` salva e carica una griglia come array JSON di righe. Ogni elemento è lo stato della cella secondo la tabella 2: 0 per una cella libera, 128 per un ostacolo.

```
[
  [0, 128,   0],
  [0,   0, 128]
]
```

Il caricamento rifiuta JSON non valido, matrici vuote o non rettangolari e valori fuori dall'intervallo $[0, 255]$, con un'eccezione esplicita invece di un risultato parziale. Nel piano sperimentale ogni griglia generata viene salvata in `risultati/griglie_json/` e ricaricata prima di eseguire CamminoMin: i 23 file prodotti sono gli input effettivi delle misure e permettono di ripetere ogni esperimento sulla stessa griglia.

3 Compito Due: contesto, complemento e distanza libera

Il calcolo è disponibile come metodo `CalcoloChiusura.calcola(griglia, x, y)` e lavora su qualunque griglia, in particolare su quelle prodotte dal generatore e salvate in JSON. Nell'applicazione viene invocato da CamminoMin a ogni chiamata; la traccia osserva che CamminoMin comprende l'elaborazione del Compito Due (pag. 34). Lo stesso calcolo è disponibile anche da solo, dalla riga di comando (sezione 3.5).

Nel seguito si usano le nozioni della traccia. Un *cammino libero* da O è privo di ostacoli ed è formato da mosse diagonali in un solo verso seguite da mosse rettilinee in un solo verso (*tipo 1*), o dalle stesse mosse in ordine inverso (*tipo 2*); i versi sono quelli del quadrante in cui si trova la destinazione. Il *contesto* di O contiene le celle raggiungibili con un cammino libero di tipo 1, il *complemento* quelle raggiungibili *solo* con un cammino di tipo 2; la loro unione è la *chiusura*. La *frontiera* è formata dalle celle della chiusura adiacenti ad almeno una cella attraversabile esterna alla chiusura.

3.1 Distanza libera

La distanza libera è la lunghezza di un cammino libero, cioè del cammino minimo in assenza di ostacoli. Posti $\Delta_x = |x_D - x_O|$, $\Delta_y = |y_D - y_O|$, $\Delta_{\min} = \min(\Delta_x, \Delta_y)$ e $\Delta_{\max} = \max(\Delta_x, \Delta_y)$, si calcola in tempo costante:

$$\mathrm{dlib}(O, D) = \sqrt{2}\,\Delta_{\min} + \Delta_{\max} - \Delta_{\min}. \tag{1}$$

Nessun cammino fra O e D può essere più corto, quindi $\mathrm{dlib}(O, D)$ è un limite inferiore della lunghezza di qualunque cammino fra le due celle. Il record `DistanzaLibera` conserva separatamente le due componenti del cammino libero, il numero di mosse rettilinee $\Delta_{\max} - \Delta_{\min}$ (componente "torre") e il numero di mosse diagonali $\Delta_{\min}$ (componente "alfiere"), insieme al valore reale della lunghezza.

Due scelte riguardano l'aritmetica in virgola mobile:

- le lunghezze sono confrontate con una tolleranza $\varepsilon = 10^{-10}$, perché somme di multipli di $\sqrt{2}$ ottenute in ordini diversi possono differire nell'ultima cifra;
- la lunghezza infinita, usata per una destinazione irraggiungibile, è rappresentata da `Double.POSITIVE_INFINITY`; la somma con un valore infinito restituisce infinito.

3.2 Calcolo di contesto e complemento

Il calcolo sfrutta la forma dei cammini liberi. Si considerino le otto semirette che partono da O nelle direzioni N, S, E, W, NE, NW, SE, SW e si fermano al primo ostacolo o al bordo della griglia; nel seguito sono chiamate *linee primarie*. Ogni cella di una linea primaria è raggiungibile con un cammino libero di sole mosse uguali, quindi di tipo 1. Le altre celle della chiusura si ottengono con un'unica deviazione:

- un cammino di tipo 1 con entrambe le componenti percorre un tratto di una linea primaria diagonale e poi una linea rettilinea in uno dei due versi che compongono la diagonale (per NE: E oppure N);
- un cammino di tipo 2 percorre un tratto di una linea primaria rettilinea e poi una linea diagonale nel quadrante corrispondente (per N: NW oppure NE).

Da ogni cella di una linea primaria partono quindi due *linee laterali*, anch'esse interrotte dal primo ostacolo o dal bordo (tabella 3). Le celle delle laterali che partono da una diagonale sono

nel contesto; quelle delle laterali che partono da una linea rettilinea sono raggiungibili con un cammino di tipo 2 e vengono marcate come complemento.

Linea primaria	**Linee laterali**	**Stato delle celle laterali**
N, S, E, W	le due diagonali del quadrante (per N: NW e NE)	`COMPLEMENTO` (tipo 2)
NE, NW, SE, SW	le due rettilinee componenti (per NE: E e N)	`CONTESTO` (tipo 1)

Tabella 3: Direzioni percorse da `CalcoloChiusura` a partire da ogni linea primaria.

Una cella può essere raggiunta sia da un cammino di tipo 1 sia da uno di tipo 2. Poiché il complemento contiene soltanto le celle raggiungibili *solo* con il tipo 2, una passata finale toglie il bit `COMPLEMENTO` alle celle che hanno anche il bit `CONTESTO`. Il contesto e il complemento sono così calcolati insieme, come la traccia consente (pag. 21), in un'unica scansione delle otto direzioni (algoritmo 2).

Ogni passo verifica soltanto la cella di arrivo. Una mossa diagonale fra due ostacoli che si toccano per uno spigolo è quindi ammessa, come richiesto dalla traccia. La fig. 1(a) mostra il risultato su una delle griglie del piano sperimentale e la tabella 4 gli stati che ne risultano: le 144 celle assumono otto valori diversi, ciascuno una combinazione dei bit della tabella 2.

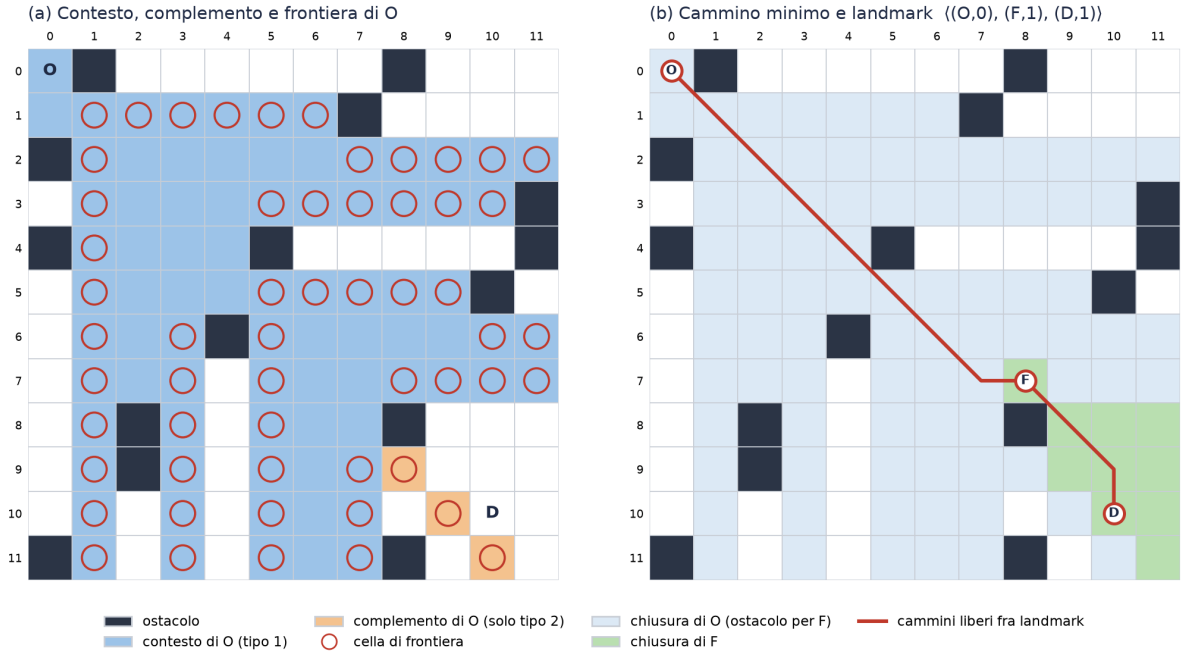


Figura 1: Griglia 12×12 con 15 ostacoli (seed 50), $O = (0, 0)$ e $D = (10, 10)$. (a) Contesto, complemento e frontiera di O calcolati con l'algoritmo 2. (b) Cammino minimo di lunghezza $9\sqrt{2}+2 \approx 14{,}728$ restituito da CamminoMin: D non appartiene alla chiusura di O, quindi il cammino passa per la cella di frontiera $F = (8, 7)$, la cui chiusura è calcolata trattando come ostacoli le celle della chiusura di O. I tratti fra landmark sono disegnati come cammini liberi a solo scopo illustrativo.

Valore	Bit	Proprietà	Celle
0	`00000000`	libera, fuori dalla chiusura	44
1	`00000001`	contesto	22
3	`00000011`	contesto, regina	6
5	`00000101`	contesto, origine	1
17	`00010001`	contesto, frontiera	51
19	`00010011`	contesto, regina, frontiera	2
24	`00011000`	complemento, frontiera	3
128	`10000000`	ostacolo	15

Tabella 4: Stati delle celle nella griglia della fig. 1(a).

Algoritmo 2 Calcolo di chiusura e frontiera di una cella O

Ingresso: griglia g di dimensioni $W \times H$, cella O
Uscita: matrice degli stati s, chiusura di O, frontiera di O

procedure CalcolaChiusura(g, O)
  $s \leftarrow$ matrice $H \times W$ con il solo bit `OSTACOLO` copiato da g
  if O è un ostacolo **then return** $s, \emptyset, \emptyset$
  for each direzione primaria d fra le otto **do**
    for each cella c di Linea(g, O, d) **do**
      $s[c] \leftarrow s[c] \mid$ `CONTESTO` $\mid$ `REGINA`
      classe $\leftarrow$ `CONTESTO` **if** d è diagonale **else** `COMPLEMENTO`
      for each direzione laterale ℓ di d (tabella 3) **do**
        for each cella c' di Linea(g, c, ℓ) **do**
          $s[c'] \leftarrow s[c'] \mid$ *classe*
  for each cella c con $s[c]$ contenente `CONTESTO` **do**
    togli `COMPLEMENTO` da $s[c]$ ▷ complemento esclusivo
  $s[O] \leftarrow s[O] \mid$ `ORIGINE` $\mid$ `CONTESTO`
  chiusura $\leftarrow \{c : s[c]$ contiene `CONTESTO` o `COMPLEMENTO`$\}$
  frontiera $\leftarrow \emptyset$
  for each $c \in$ *chiusura* **do**
    if esiste un vicino v di c nella griglia, non ostacolo e non in *chiusura* **then**
      $s[c] \leftarrow s[c] \mid$ `FRONTIERA`; aggiungi c a *frontiera*
  return s, *chiusura*, *frontiera*

function Linea(g, c, d) ▷ celle da c (esclusa) in direzione d
  $L \leftarrow \langle\rangle$; $c \leftarrow c + d$
  while c è nella griglia **and** c non è un ostacolo **do**
    aggiungi c a L; $c \leftarrow c + d$
  return L

3.3 Frontiera

Una cella della chiusura è di frontiera se almeno una delle otto celle adiacenti è dentro la griglia, non è un ostacolo e non appartiene alla chiusura. Le celle esterne alla griglia non contano come celle attraversabili: se la chiusura copre tutta una griglia senza ostacoli, la frontiera è vuota. Il tipo di una cella di frontiera si legge dal suo stato: $t = 1$ se ha il bit `CONTESTO`, $t = 2$ altrimenti.

3.4 Complessità

Sia $n = WH$ il numero di celle.

Tempo. La copia iniziale degli ostacoli, la passata che rende esclusivo il complemento e la raccolta della chiusura scandiscono l'intera griglia: $\Theta(n)$. Le linee primarie hanno lunghezza complessiva $O(W + H)$. Le linee laterali che partono da una stessa linea primaria nella stessa direzione sono parallele, quindi disgiunte; ogni cella può essere attraversata al più da una laterale per ciascuna delle due linee primarie che delimitano il suo ottante. Le laterali visitano perciò $O(n)$ celle in totale. Il controllo della frontiera esamina otto vicini per ogni cella della chiusura, $O(n)$. Complessivamente il calcolo costa $\Theta(n)$, anche quando la chiusura è piccola, perché le scansioni coprono sempre tutta la griglia.

Spazio. La matrice degli stati occupa $4n$ byte (un `int` per cella); la lista della chiusura e l'array della frontiera hanno al più n elementi. Lo spazio è quindi $\Theta(n)$.

Il costo lineare nella dimensione della griglia ha conseguenze sul Compito Tre: CamminoMin calcola una chiusura a ogni chiamata ricorsiva non servita dalla cache, quindi il suo costo è almeno proporzionale a n per il numero di chiusure calcolate (sezione 5.1).

3.5 Applicazione

L'opzione `--chiusura` esegue il solo Compito Due su una griglia generata con i parametri della sezione 2 oppure letta da un file JSON prodotto dal generatore (`--grid-input`). Per la cella O indicata l'applicazione stampa la mappa della chiusura, il numero di celle di contesto, complemento e frontiera, l'appartenenza di D alla chiusura e la distanza libera $\mathrm{dlib}(O, D)$, scomposta in mosse rettilinee e diagonali; con `--chiusura-output` salva anche gli stati di tutte le celle in JSON, nel formato della tabella 2. Sulla griglia della fig. 1(a) l'uscita è la seguente (le celle di frontiera sono in maiuscolo) e i conteggi coincidono con la tabella 4.

```
java -jar asd-project.jar --grid-input
    risultati/griglie_json/scenario_02_12x12_15ost_seed50.json --origin 0,0
    --destination 10,10 --chiusura
O   #   .   .   .   .   .   .   #   .   .   .
c   C   C   C   C   C   C   #   .   .   .   .
#   C   c   c   c   c   c   C   C   C   C   C
.   C   c   c   c   C   C   C   C   C   C   #
#   C   c   c   c   #   .   .   .   .   .   #
.   C   c   c   c   C   C   C   C   C   #   .
.   C   c   C   #   C   c   c   c   c   C   C
.   C   c   C   .   C   c   c   C   C   C   C
.   C   #   C   .   C   c   c   #   .   .   .
.   C   #   C   .   C   c   C   K   .   .   .
.   C   .   C   .   C   c   C   .   K   D   .
#   C   .   C   .   C   c   C   #   .   K   .
Legenda: O origine, D destinazione, # ostacolo, c contesto (tipo 1), k complemento
    (tipo 2), maiuscole = frontiera, . fuori dalla chiusura
Contesto: 82 celle | complemento: 3 celle | frontiera: 56 celle (53 di tipo 1, 3 di
    tipo 2)
O=(0,0), D=(10,10): D si trova fuori dalla chiusura di O
dlib(O,D) = 0 + 10*sqrt(2) = 14.142136 (0 mosse rettilinee, 10 diagonali)
```

3.6 Verifica

Oltre ai test unitari su casi costruiti a mano, `CalcoloChiusura` è confrontato con un *oracolo*, cioè un'implementazione di riferimento che calcola lo stesso risultato con un metodo diverso e più semplice, così che un errore difficilmente sia comune ai due. L'oracolo (`CalcoloChiusuraOracleTest`) non usa linee primarie e laterali: per ogni cella c percorre l'unico cammino di tipo 1 e l'unico di tipo 2 da O a c e controlla che non attraversino ostacoli, applicando direttamente la definizione; la frontiera è ricavata dalla chiusura così ottenuta. Il test enumera tutte le $2^9 = 512$ disposizioni di ostacoli su una griglia 3×3 e, per ogni origine libera, richiede che contesto, complemento, chiusura e frontiera coincidano cella per cella. Un test dedicato verifica che la mossa diagonale fra due ostacoli adiacenti per uno spigolo sia ammessa e costi $\sqrt{2}$.

4 Compito Tre: l'algoritmo CamminoMin

4.1 Idea e pseudocodice di riferimento

Se D appartiene alla chiusura di O, un cammino minimo è il cammino libero fra le due celle e la sua lunghezza è dlib(O, D). Altrimenti, qualunque cammino da O a D esce dalla chiusura di O passando per una cella della sua frontiera. La traccia definisce quindi ricorsivamente il cammino minimo come il più corto fra i cammini ottenuti componendo il cammino libero da O a una cella di frontiera F con un cammino minimo da F a D, al variare di F nella frontiera. Nella chiamata su F le celle della chiusura di O sono trattate come ostacoli, così il cammino non ripassa da una regione già coperta.

L'algoritmo 3 riporta lo pseudocodice della traccia (pag. 30); la numerazione delle righe è quella usata nel resto della relazione.

Algoritmo 3 CamminoMin come definito nella traccia [1]

```
 1: procedure CamminoMin(O, D, dim, osta)
 2:     ▷ dim: dimensioni della griglia; osta: celle non attraversabili
 3:     contesto ← CalcolaContesto(O, dim, osta)
 4:     if D ∈ contesto then
 5:         return dlib(O, D), ⟨(O, 0), (D, 1)⟩
 6:     complemento ← CalcolaComplemento(O, dim, osta)
 7:     if D ∈ complemento then
 8:         return dlib(O, D), ⟨(O, 0), (D, 2)⟩
 9:     frontiera ← CalcolaFrontiera(contesto, complemento, dim, osta)
10:     if frontiera = ∅ then                                   ▷ vicolo cieco
11:         return ∞, ⟨⟩
12:     lunghezzaMin ← ∞
13:     seqMin ← ⟨⟩
14:     for each (F, t) ∈ frontiera do
15:         lF ← dlib(O, F)
16:         if lF < lunghezzaMin then
17:             ▷ una condizione più forte è lF + dlib(F, D) < lunghezzaMin
18:             lFD, seqFD ← CamminoMin(F, D, dim, osta ∪ contesto ∪ complemento)
19:             lTot ← lF + lFD
20:             if lTot < lunghezzaMin then
21:                 lunghezzaMin ← lTot
22:                 seqMin ← Compatta(⟨(O, 0), (F, t)⟩, seqFD)
23:     return lunghezzaMin, seqMin
```

4.2 Implementazione

La classe `CamminoMin` implementa l'algoritmo 4. Il metodo pubblico `calcola(griglia, O, D)` restituisce un `Cammino`, formato dalla lunghezza (`DistanzaLibera`) e dalla lista dei landmark. Due sovraccarichi ricevono anche un oggetto `StatisticheEsecuzione`, in cui accumulare i contatori del riassunto, e la variante dell'algoritmo da eseguire (sezione 4.6); senza indicazioni si usa la variante `base`.

Senza limite propagato L vale sempre ∞ e l'algoritmo coincide con la versione della traccia arricchita da ordinamento, riga 17 e cache.

La tabella 5 collega le righe della traccia all'implementazione. La traccia ammette una codifica diversa dallo pseudocodice purché produca gli stessi effetti (pag. 34–35).

Algoritmo 4 CamminoMin come implementato in `CamminoMin`

Ingresso: griglia g, celle attraversabili O e D, limite L (∞ nella chiamata principale), tabella *cache* (vuota alla prima chiamata)

Uscita: lunghezza del cammino minimo da O a D e sequenza dei landmark, oppure $(\infty, \langle\rangle)$ se non esiste un cammino più corto di L

1: **function** CamminoMin(g, O, D, L)
2: $k \leftarrow$ Chiave(ostacoli di g, O, D) ▷ se la cache è attiva
3: **if** *cache*$[k]$ è esatto, o esclude cammini più corti di un limite $\geq L$ **then**
4: **return** *cache*$[k]$ ▷ $(\infty, \langle\rangle)$ nel secondo caso
5: s, *chiusura*, *frontiera* $\leftarrow$ CalcolaChiusura(g, O) ▷ algoritmo 2
6: **if** $s[D]$ contiene `CHIUSURA` **then** ▷ righe 4–8, anche $O = D$
7: $r \leftarrow (\text{dlib}(O, D), \langle (O, 0), (D, \text{Tipo}(s[D])) \rangle)$
8: *cache*$[k] \leftarrow r$ (esatto); **return** r
9: *lunghezzaMin* $\leftarrow L$; *seqMin* $\leftarrow \langle\rangle$
10: ordina *frontiera* per $\text{dlib}(O, F) + \text{dlib}(F, D)$ crescente ▷ tranne `senza-ordinamento`
11: **for each** $F \in$ *frontiera* **do**
12: $lF \leftarrow \text{dlib}(O, F)$
13: **if** $lF \geq$ *lunghezzaMin* **then continue** ▷ riga 16 falsa
14: **if** $lF + \text{dlib}(F, D) \geq$ *lunghezzaMin* **then continue** ▷ riga 17 falsa
15: $g' \leftarrow g$ con le celle di *chiusura* $\setminus \{F\}$ come ostacoli
16: $L' \leftarrow$ *lunghezzaMin* $- lF$ se il limite è propagato, altrimenti ∞
17: lFD, *seqFD* $\leftarrow$ CamminoMin(g', F, D, L')
18: **if** $lF + lFD <$ *lunghezzaMin* **then**
19: *lunghezzaMin* $\leftarrow lF + lFD$
20: *seqMin* $\leftarrow \langle (O, 0), (F, \text{Tipo}(s[F])) \rangle \cdot$ *seqFD*[2..] ▷ Compatta
21: **if** *seqMin* $= \langle\rangle$ **then** ▷ nessun cammino più corto di L
22: *cache*$[k] \leftarrow$ "nessun cammino più corto di L"; **return** $(\infty, \langle\rangle)$
23: *cache*$[k] \leftarrow$ (*lunghezzaMin*, *seqMin*) (esatto)
24: **return** *lunghezzaMin*, *seqMin*

Righe della traccia	Implementazione
3, 6, 9	Un'unica chiamata a `CalcoloChiusura` produce contesto, complemento, frontiera e tipo di ogni cella (sezione 3.2).
4–8	Il caso base controlla la maschera `CHIUSURA` nello stato di D; il tipo di D è 1 se ha il bit `CONTESTO`, 2 altrimenti. Se $O = D$, O è nel proprio contesto e il risultato è $\big(0, \langle (O,0), (O,1)\rangle\big)$, come richiesto a pag. 32.
10–11	Con frontiera vuota il ciclo non esegue iterazioni e la funzione restituisce $(\infty, \langle\rangle)$, cioè lo stesso risultato.
12–13	*lunghezzaMin* parte da ∞, oppure dal limite ricevuto se il limite è propagato (sezione 4.5).
14	La frontiera è visitata in ordine di limite inferiore crescente (ordine di scansione nella variante `senza-ordinamento`). O non è mai di frontiera: i suoi vicini attraversabili sono le prime celle delle linee primarie e appartengono al contesto.
16–17	La condizione della riga 16 è controllata per prima, poi quella più forte suggerita alla riga 17. I due esiti negativi sono contati separatamente (sezione 4.7).
18	Gli ostacoli della chiamata ricorsiva sono quelli di g più la chiusura di O tranne F (sezione 4.3).
20–22	Il confronto è stretto: a parità di lunghezza resta il primo cammino trovato. Compatta sostituisce il primo landmark $(F, 0)$ di *seqFD* con (F, t).

Tabella 5: Corrispondenza fra lo pseudocodice della traccia e l'implementazione.

4.3 Ostacoli nelle chiamate annidate

Secondo la traccia (pag. 28 e 32), nella chiamata su F tutte le celle della chiusura di O diventano ostacoli, compresa F, che però resta l'origine della nuova chiusura. L'implementazione ottiene lo stesso effetto escludendo F dall'insieme di ostacoli aggiunti: le celle raggiungibili da F sono le stesse, e `CalcoloChiusura` non deve gestire un'origine marcata come ostacolo. La nuova griglia g' è una vista immutabile (sezione 2.3): ogni ramo ricorsivo accumula i propri ostacoli senza modificare la griglia dei rami fratelli.

4.4 Sequenza dei landmark

La sequenza dei landmark $\langle (O,0), (F_1,t_1), \ldots, (D,t_D)\rangle$ elenca l'origine e le celle di frontiera attraversate dal cammino; il tipo di un landmark è quello del cammino libero che lo raggiunge dal landmark precedente, quindi la sequenza descrive il cammino in modo univoco. Nell'implementazione un landmark è un record con le coordinate e il tipo $t \in \{0,1,2\}$; il tipo 0 identifica l'origine della chiamata. La lista dei landmark di un `Cammino` è immutabile. Se D è irraggiungibile il risultato è la lunghezza infinita con la sequenza vuota.

Nell'esempio della fig. 1(b) la chiamata da $O = (0,0)$ restituisce $\langle (O,0), ((8,7),1), (D,1)\rangle$. La chiamata nel verso opposto, da $(10,10)$ a $(0,0)$, restituisce $\langle ((10,10),0), ((0,1),2), ((0,0),1)\rangle$: un cammino diverso, ma della stessa lunghezza $9\sqrt{2}+2$. In entrambi i casi la somma delle distanze libere fra landmark consecutivi è uguale alla lunghezza del cammino; i test di confronto con Dijkstra (sezione 5.3) verificano questa proprietà su ogni cammino finito, insieme al fatto che ogni tratto sia realizzabile come cammino libero.

4.5 Ottimizzazioni

Le ottimizzazioni riducono il numero di chiamate ricorsive senza cambiare la lunghezza restituita; il loro effetto è misurato nella sezione 5.5.

Ordinamento della frontiera. Le celle di frontiera sono visitate in ordine crescente di $\mathrm{dlib}(O, F) + \mathrm{dlib}(F, D)$, un limite inferiore della lunghezza di qualunque cammino da O a D che passi per F. I candidati più promettenti vengono esplorati per primi, *lunghezzaMin* scende presto e le condizioni di potatura scartano più rami. L'ordinamento costa $O(f \log f)$, con f numero di celle di frontiera; le due distanze libere di ogni candidato sono calcolate una volta sola e riusate dalla potatura.

Condizione della riga 17. Dopo la riga 16 viene controllata la condizione più forte $lF + \mathrm{dlib}(F, D) < \mathit{lunghezzaMin}$. La potatura è corretta perché $\mathrm{dlib}(F, D)$ non supera la lunghezza di nessun cammino da F a D: se la condizione è falsa, passare per F non può dare un cammino strettamente più corto di quello già trovato. Il confronto sperimentale fra le due condizioni, facoltativo per i gruppi di tre persone (pag. 67), non fa parte di questo progetto.

Memoizzazione. Il risultato di una chiamata dipende dagli ostacoli della griglia, da O, da D e, con il limite propagato, dal limite ricevuto. Il risultato viene salvato in una `HashMap` la cui chiave è formata dal `BitSet` degli ostacoli della vista (sezione 2.3) e dalle coordinate di O e D: la chiave occupa $n/8$ byte e `equals` e `hashCode` costano $O(n/64)$. La chiave è esatta, perché un hash intero degli ostacoli potrebbe collidere e restituire il risultato di un altro sottoproblema. Una stringa con le coordinate di tutti gli ostacoli darebbe chiavi equivalenti, ma richiederebbe una scansione della griglia e decine di caratteri per ostacolo. Le direzioni $O \to D$ e $D \to O$ usano tabelle separate, così la verifica di simmetria confronta due calcoli indipendenti.

Limite propagato. Nella versione della traccia ogni chiamata annidata cerca il cammino minimo da F a D partendo da $\mathit{lunghezzaMin} = \infty$, anche quando il chiamante ha già un cammino di lunghezza *lunghezzaMin* e gli serve soltanto sapere se ne esiste uno più corto di $\mathit{lunghezzaMin} - \mathrm{dlib}(O, F)$. La variante con limite propagato passa questo valore L' alla chiamata annidata, che inizializza con esso il proprio *lunghezzaMin*: le righe 16 e 17 potano così anche all'interno della ricorsione. Una chiamata con limite L restituisce il cammino minimo del sottoproblema se è più corto di L, altrimenti $(\infty, \langle\rangle)$. Il risultato resta corretto: un cammino trovato sotto il limite viene confrontato con tutti i candidati che potrebbero batterlo, ed è quindi il minimo; un ramo che non trova nulla sotto il limite non può migliorare il cammino del chiamante. La cache distingue i due casi: un risultato trovato è esatto e vale per qualunque limite; un risultato vuoto significa soltanto "nessun cammino più corto di L" e viene riusato solo per chiamate con limite non superiore a L.

Viste immutabili. La griglia g' della chiamata ricorsiva richiede la copia di un `BitSet` di n bit e l'inserimento delle celle della chiusura, invece della copia di una matrice di n interi.

4.6 Varianti

L'enumerazione `VarianteCamminoMin` attiva o disattiva le ottimizzazioni, per confrontarle nella sperimentazione (tabella 6). La riga 17 è attiva in tutte le varianti. La variante si sceglie da riga di comando con `--variante`.

Variante	Cache	Ordinamento	Limite propagato
`base`	sì	sì	no
`senza-cache`	no	sì	no
`senza-ordinamento`	sì	no, ordine di scansione della griglia	no
`limite-propagato`	sì	sì	sì

Tabella 6: Varianti di CamminoMin. Tutte restituiscono la stessa lunghezza minima.

4.7 Contatori e riassunto

La traccia chiede, per ogni istanza e su richiesta dell'utente, un riassunto con dimensioni e tipo della griglia, numero totale di celle di frontiera, numero di volte in cui la condizione della riga 16/17 è risultata falsa e prestazioni (pag. 71). `StatisticheEsecuzione` raccoglie i contatori della tabella 7 durante l'esecuzione. Un'istanza comprende sempre entrambe le direzioni $O \to D$ e $D \to O$; i contatori sommano le due direzioni e tutte le chiamate ricorsive.

Campo	Significato
`front`	celle di frontiera di tutte le chiusure calcolate (le chiamate servite dalla cache non calcolano chiusure)
`r16False`	volte in cui la condizione della riga 16 è risultata falsa
`r17Prune`	volte in cui la riga 16 era vera ma la condizione più forte della riga 17 era falsa
`pruneTot`	somma dei due precedenti: volte in cui la condizione effettivamente usata è risultata falsa e il ramo è stato potato
`variante`	variante di CamminoMin eseguita (tabella 6)
`chiamate`	invocazioni di CamminoMin, comprese quelle servite dalla cache
`cacheHit`	invocazioni servite dalla cache, che non calcolano una chiusura
`prof`	profondità massima della ricorsione
`chiusurePicco`	massimo numero di matrici di chiusura attive contemporaneamente lungo la pila delle chiamate
`memEst`	stima della memoria di tali matrici: `chiusurePicco` $\cdot\, 4n$ byte
`tempoNs`	tempo minimo, mediano e massimo delle ripetizioni (sezione 5.2)
`seqOD`, `seqDO`	sequenze dei landmark nei due versi

Tabella 7: Contatori del riassunto. L'elenco completo delle colonne del CSV è nell'appendice A.

Poiché $\mathrm{dlib}(F, D) \geq 0$, quando la condizione della riga 16 è falsa lo è anche quella della riga 17. `pruneTot` conta quindi le volte in cui la condizione della riga 17 è risultata falsa, e `r16False` quante di queste sarebbero state potate anche dalla sola riga 16. La profondità e le chiusure attive possono differire: una chiamata servita dalla cache aumenta la profondità ma non alloca una nuova matrice.

Il riassunto è stampato su una riga per istanza e variante, con le lunghezze arrotondate a quattro decimali (il CSV le conserva con sei); nel piano è salvato anche in `risultati/riassunti.txt`. Quello dell'istanza della fig. 1, qui spezzato ai separatori per leggibilità, è:

```
[G12x12_d10_s50_p1] variante=base | tipo=Casuale | 12x12 | ost=15
  | O=(0,0)->D=(10,10) | OD=14.7279 | DO=14.7279 | corr=true
  | front=118 | r16False=1 | r17Prune=96 | pruneTot=97
  | chiamate=18 | cacheHit=2 | prof=4 | chiusurePicco=4 | memEst=2304B
  | tempoNs[min/med/max]=180900/184000/218900 | warmup=1 | rip=5
  | seqOD=[(x=0,y=0,tipo=0), (x=8,y=7,tipo=1), (x=10,y=10,tipo=1)]
  | seqDO=[(x=10,y=10,tipo=0), (x=0,y=1,tipo=2), (x=0,y=0,tipo=1)]
```

5 Compito Quattro: analisi e sperimentazione

5.1 Analisi teorica

Nel seguito $n = WH$ è il numero di celle della griglia, C il numero totale di invocazioni di CamminoMin (comprese quelle servite dalla cache), $C_c \leq C$ il numero di invocazioni che calcolano una chiusura, f_i il numero di celle di frontiera dell'invocazione i e l la profondità massima della ricorsione.

Tempo di una invocazione. Un'invocazione cerca il proprio sottoproblema nella cache: la chiave riusa il `BitSet` degli ostacoli della vista, quindi `hashCode` ed `equals` costano $O(n/64)$. Se la chiave è presente si ferma. Altrimenti calcola la chiusura, $\Theta(n)$ (sezione 3.4), e ordina la frontiera, $O(f_i \log f_i)$. Per ogni cella di frontiera non potata costruisce la griglia della chiamata ricorsiva, $O(n)$; questo costo si può attribuire all'invocazione figlia. Poiché $f_i \leq n$, il tempo totale è

$$T = \sum_{i=1}^{C_c} O(n + f_i \log f_i) + O(C \cdot n/64) = O(C_c \cdot n \log n + C \cdot n), \qquad T = \Omega(C_c \cdot n). \quad (2)$$

Il prodotto $C_c \cdot n$, chiamato *lavoro stimato* nel seguito, è quindi la grandezza che dovrebbe guidare il tempo misurato.

Caso migliore. Se D appartiene alla chiusura di O basta una invocazione per verso e il costo è $\Theta(n)$. È il caso di 28 delle 67 istanze del piano sperimentale; in una ventinovesima, sulla griglia 5×5 quasi piena, la frontiera di O è vuota e il risultato ∞ si ottiene ugualmente con una sola invocazione per verso.

Caso peggiore. Ogni chiamata annidata aggiunge agli ostacoli almeno la cella da cui parte la chiusura precedente, quindi lungo un ramo il numero di celle libere diminuisce strettamente e $l \leq n$. Ogni invocazione ha al più $f_i \leq n$ figli. Il numero di invocazioni è quindi limitato da una quantità esponenziale, dell'ordine di n^l; questa analisi non fornisce un limite polinomiale in n, e la sperimentazione mostra una crescita combinatoria. Ordinamento della frontiera, potatura, cache e limite propagato riducono C in pratica ma non cambiano il caso peggiore. Nel piano, con la variante `base`, C arriva a 102 202 su una griglia di 400 celle (435 793 senza cache) e nella serie sui limiti di scala a 508 161 (sezione 5.6).

Spazio. Ogni invocazione attiva che non è servita dalla cache tiene in memoria la propria matrice degli stati ($4n$ byte), la chiusura e la frontiera ($O(n)$) e la griglia del figlio in esecuzione (n bit). La pila delle chiamate occupa quindi $O(l \cdot n)$. La cache conserva un elemento per ogni sottoproblema risolto: la chiave contiene un `BitSet` di n bit e il valore una sequenza di al più $l + 1$ landmark. Lo spazio totale è

$$S = O(l \cdot n) + O\big(C_c \cdot (n/8 + l)\big) \text{ byte}, \quad (3)$$

dominato dalla cache quando C_c cresce. La stima `memEst` del riassunto conta soltanto le matrici della pila (`chiusurePicco`$\cdot 4n$ byte) e va letta come limite inferiore della memoria effettivamente usata.

5.2 Piano sperimentale e protocollo

Il piano comprende 23 griglie casuali e 67 istanze, cioè coppie (O, D) (tabella 8). Le dimensioni vanno da 12×12 a 50×50; per ogni dimensione c'è una griglia senza ostacoli, che isola il costo

del caso base, e le griglie con ostacoli sono generate da seed consecutivi. Il peso del piano è sulle griglie 20×20 con densità fra il 5% e il 20%, dove la ricorsione è significativa ma ogni variante termina in tempi brevi. Le istanze più difficili sono coperte dalla serie sui limiti di scala (sezione 5.6).

Ogni istanza viene eseguita con più varianti di CamminoMin (tabella 6). Le griglie senza ostacoli e il caso irraggiungibile usano solo `base` e `limite-propagato`, perché basta una chiamata per verso. Le griglie con ostacoli usano anche `senza-cache` e, dove termina in tempi accettabili, `senza-ordinamento`. In prove preliminari, sulle griglie 20×20 al 20% una sola esecuzione senza ordinamento richiedeva da alcuni secondi a qualche decina di secondi e milioni di chiamate: per mantenere il piano entro pochi minuti la variante è stata esclusa da quel gruppo e dalle griglie 30×30 e 50×50. In totale il piano comprende 215 esecuzioni (istanza e variante).

Dimensione	Ostacoli	Densità	Seed	Griglie	Istanze	Varianti
12×12	0	0%	50	1	3	B, L
12×12	15	10,4%	50–52	3	9	tutte
20×20	0	0%	100	1	3	B, L
20×20	20	5%	100–102	3	9	tutte
20×20	40	10%	100–102	3	9	tutte
20×20	80	20%	100–102	3	9	B, C, L
30×30	0	0%	200	1	3	B, L
30×30	45	5%	200–202	3	9	B, C, L
50×50	0	0%	300	1	3	B, L
50×50	50	2%	300–302	3	9	B, C, L
5×5	23	92%	1	1	1	B, L
Totale				**23**	**67**	

Tabella 8: Griglie e istanze del piano sperimentale. Varianti: B = `base`, C = `senza-cache`, L = `limite-propagato`; "tutte" comprende anche `senza-ordinamento`.

Su ogni griglia si provano tre coppie, definite come frazioni delle coordinate massime: dall'angolo in alto a sinistra a quello in basso a destra (5% → 95%), dall'angolo in alto a destra a quello in basso a sinistra, e dal centro verso l'alto a destra (50% → (90%, 10%)). Se la cella indicata è un ostacolo si usa la cella libera più vicina in distanza di Chebyshev. La griglia 5×5 con 23 ostacoli ha una sola coppia, scelta in modo che D sia irraggiungibile, per verificare il risultato $(\infty, \langle\rangle)$ anche nel piano e non solo nei test.

Prima delle misure l'intero piano viene eseguito una volta, senza registrare i tempi, per completare la compilazione JIT della JVM. Poi, per ogni istanza e variante:

1. si esegue una ripetizione di riscaldamento, esclusa dalle misure;
2. seguono 5 ripetizioni misurate; ciascuna misura con `System.nanoTime()` il tempo complessivo di CamminoMin(O, D) e CamminoMin(D, O);
3. il tempo dell'esecuzione è la mediana delle 5 misure; si registrano anche minimo e massimo;
4. i contatori, che non dipendono dal tempo, devono coincidere in tutte le ripetizioni, altrimenti l'esecuzione si interrompe con un errore; si conservano quelli della ripetizione mediana.

Le misure sono state raccolte l'8 novembre 2026 su un AMD Ryzen 7 5800X (8 core, 16 thread) con 31,9 GiB di RAM, Windows 11 Pro e Eclipse Temurin 21.0.6. L'algoritmo è

sequenziale e usa un solo thread. Il piano completo, compreso il passaggio di riscaldamento, richiede meno di un minuto.

5.3 Verifica della correttezza

Simmetria e coerenza fra varianti. Per ogni istanza la traccia chiede di confrontare le lunghezze calcolate nei due versi. Tutte le 215 esecuzioni superano il controllo: 66 istanze hanno lunghezze finite che differiscono meno di 10^{-10}, l'istanza 5×5 ha lunghezza infinita e sequenza vuota in entrambi i versi. Lo script di analisi verifica inoltre che su ogni istanza tutte le varianti eseguite restituiscano la stessa lunghezza. Le sequenze dei landmark nei due versi sono in generale diverse, come nell'esempio della fig. 1.

Oracolo indipendente. La simmetria non rileva un errore che colpisca allo stesso modo entrambi i versi. Per questo un test confronta CamminoMin, in tutte e quattro le varianti, con l'algoritmo di Dijkstra [2] sul grafo delle celle libere, con archi di costo 1 e $\sqrt{2}$. Le istanze sono 500 griglie 5×5 con 0–20 ostacoli e coppie (O, D) estratte a caso da un seed fisso. Il test richiede che:

- le due lunghezze coincidano, oppure siano entrambe infinite con sequenza vuota;
- la sequenza dei landmark inizi con $(O, 0)$, termini in D e abbia tipi 1 o 2 su celle libere;
- per ogni coppia di landmark consecutivi la distanza libera coincida con la distanza di Dijkstra sulla griglia originale, cioè il tratto sia realizzabile come cammino libero;
- la somma dei tratti sia uguale alla lunghezza restituita.

Griglie strutturate. Le griglie 5×5 producono ricorsioni poco profonde. Altri tre test applicano lo stesso confronto con Dijkstra, in entrambi i versi e per tutte le varianti, a griglie in cui il cammino deve attraversare molti landmark:

- una *serpentina* 15×15, con muri orizzontali e varchi alternati a destra e a sinistra: il cammino ha 8 landmark e la ricorsione arriva a profondità 8 con 90 chiamate, 46 delle quali servite dalla cache;
- una *spirale* 13×13 con un corridoio largo una cella: 13 landmark e profondità 12;
- 50 griglie casuali 10×10 con il 20% di ostacoli.

Regressione. Un test fissa i contatori della variante `base` sull'istanza della fig. 1, un altro verifica che il generatore produca le stesse griglie di un'implementazione di riferimento che aggiunge gli ostacoli uno alla volta, e un terzo che il limite propagato riduca le chiamate senza cambiare la lunghezza sull'istanza più costosa del piano.

Suite di test. Insieme all'oracolo della chiusura (sezione 3) e ai test su caso $O = D$, destinazione irraggiungibile, persistenza JSON, immutabilità della griglia nella ricorsione, contatori e riga di comando, la suite comprende 68 test JUnit, tutti superati con `mvn clean package` su JDK 21. I test dimostrano la correttezza sui casi verificati, non per ogni griglia possibile.

5.4 Risultati del piano

La tabella 9 riassume le 67 istanze per dimensione e densità per le due varianti principali. I tempi sono le mediane delle singole istanze, in millisecondi, e comprendono entrambi i versi.

			base			limite-propagato		
Dimensione	**Densità**	**Istanze**	**Mediana ms**	**Max ms**	**Max chiamate**	**Mediana ms**	**Max ms**	**Max chiamate**
5×5	92%	1	0,003 6	0,003 6	2	0,001 9	0,001 9	2
12×12	0%	3	0,026 5	0,027 4	2	0,027 2	0,042 7	2
12×12	10,4%	9	0,052 3	0,184 0	30	0,033 4	0,178 4	29
20×20	0%	3	0,053 9	0,055 0	2	0,054 1	0,087 7	2
20×20	5%	9	0,142 6	0,762 9	154	0,081 2	0,805 4	110
20×20	10%	9	0,131 3	2,155 7	536	0,128 7	0,993 6	248
20×20	20%	9	0,517 3	399,009 7	102 202	0,468 3	2,116 6	719
30×30	0%	3	0,140 2	0,140 2	2	0,141 8	0,173 1	2
30×30	5%	9	0,537 2	4,826 5	1 124	0,646 3	4,925 9	1 124
50×50	0%	3	0,340 6	0,396 4	2	0,417 4	0,514 3	2
50×50	2%	9	0,582 0	10,436 8	558	0,506 2	6,256 1	282

Tabella 9: Risultati aggregati del piano sperimentale per dimensione e densità. Il dettaglio di ogni esecuzione è in `risultati/esperimenti.csv` e `risultati/riassunti.txt`.

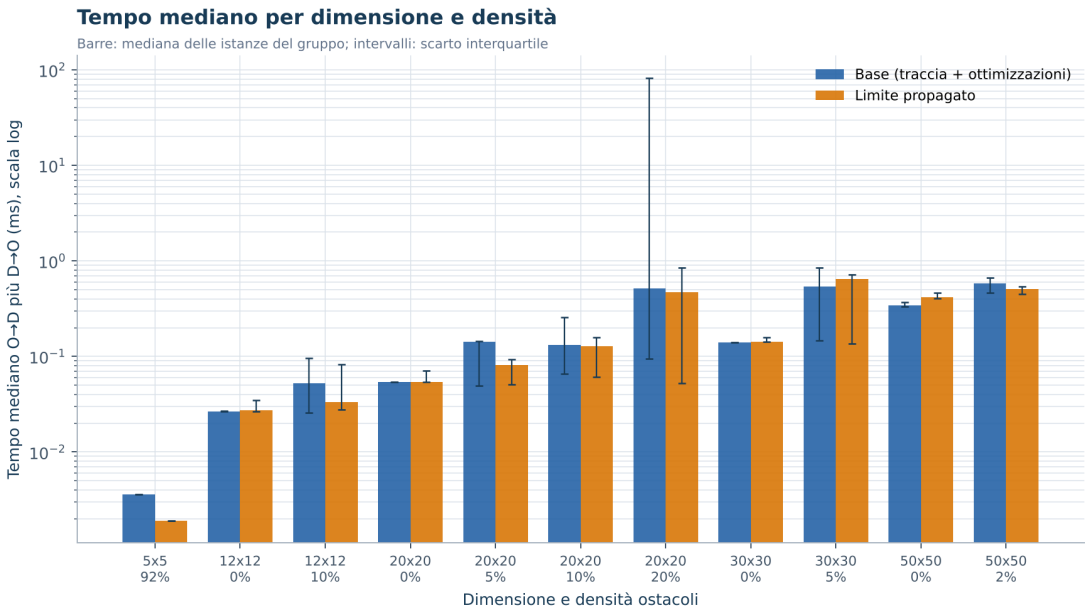


Figura 2: Tempo mediano $O \to D$ più $D \to O$ per dimensione e densità, varianti `base` e `limite-propagato` (scala logaritmica). Le barre d'errore sono l'intervallo interquartile delle istanze del gruppo.

Dalla tabella e dalla fig. 2 emergono tre aspetti.

- **Senza ostacoli il tempo cresce con la dimensione.** In tutte le 12 istanze senza ostacoli D è nella chiusura di O e basta una chiamata per verso. Il tempo della variante `base` sale da 0,027 ms a 12×12 a 0,34 ms a 50×50: le celle aumentano di 17 volte e il tempo di 13, come atteso da un calcolo di chiusura di costo $\Theta(n)$. Il passaggio di riscaldamento evita che la prima griglia misurata risulti più lenta di griglie più grandi a causa della compilazione JIT.
- **Il costo dipende dalle chiamate più che dalla dimensione.** Il gruppo più costoso per la variante `base` non è il 50×50 ma il 20×20 con il 20% di ostacoli, dove una singola

istanza richiede 102 202 chiamate e 399 ms, circa due terzi del tempo della variante `base` sull'intero piano. Le istanze di uno stesso gruppo differiscono fino a quattro ordini di grandezza, perché la disposizione degli ostacoli e la posizione di O e D decidono quanti rami sopravvivono alla potatura.

- **Il limite propagato elimina le code.** Le mediane delle due varianti sono simili, perché la maggior parte delle istanze è facile per entrambe; il massimo del gruppo 20×20 al 20% scende però da 399 ms a 2,1 ms e da 102 202 a 719 chiamate.

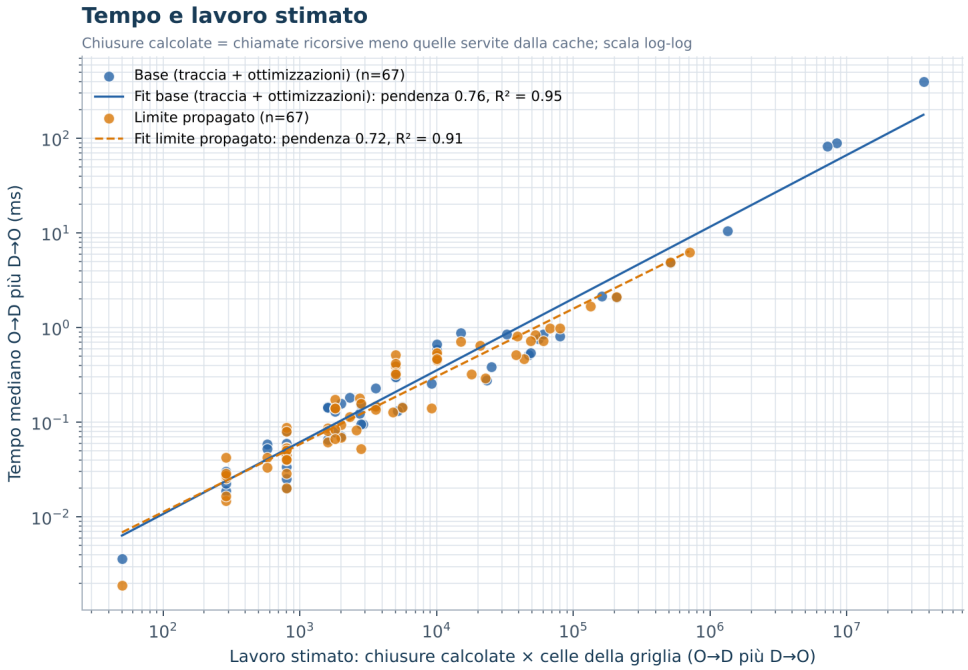


Figura 3: Tempo mediano rispetto al lavoro stimato $C_c \cdot n$ per le 67 istanze, varianti `base` e `limite-propagato`, in scala log-log, con le rette di regressione.

Che cosa determina il tempo. La tabella 10 confronta quattro grandezze come predittori del tempo della variante `base`. Il numero di chiamate da solo spiega il tempo in modo moderato, perché le chiamate servite dalla cache costano poco e perché lo stesso numero di chiusure costa di più su una griglia più grande. Il lavoro stimato $C_c \cdot n$ della eq. (2), cioè le chiusure effettivamente calcolate moltiplicate per le celle, spiega invece il 95% della varianza dei logaritmi (fig. 3); la relazione vale anche per il limite propagato ($R^2 = 0{,}909$, $\rho = 0{,}955$). La pendenza minore di 1 indica che sulle istanze più piccole pesano costi fissi, come l'allocazione dei risultati e le misure stesse, che non crescono con il lavoro.

Predittore	**Istanze**	**Spearman ρ**	**Pendenza log-log**	**R^2**
chiamate ricorsive C	67	0,737	0,704	0,753
chiusure calcolate C_c	67	0,740	0,737	0,767
celle di frontiera visitate	54	0,905	1,260	0,901
lavoro stimato $C_c \cdot n$	67	0,957	0,759	0,953

Tabella 10: Grandezze che spiegano il tempo mediano della variante `base`; per la frontiera si escludono le 13 istanze in cui è vuota. Le tre coppie di una stessa griglia non sono osservazioni indipendenti: i valori descrivono il piano e non permettono inferenze generali.

Potatura. Sull'intero piano, con la variante `base`, la condizione della riga 16 è risultata falsa 3 423 volte; in altri 20 483 casi la riga 16 era vera e il ramo è stato potato dalla condizione più forte della riga 17. Il dato descrive l'esecuzione con entrambe le condizioni attive e non equivale al confronto fra le due condizioni, che richiederebbe di eseguire l'algoritmo con la sola riga 16.

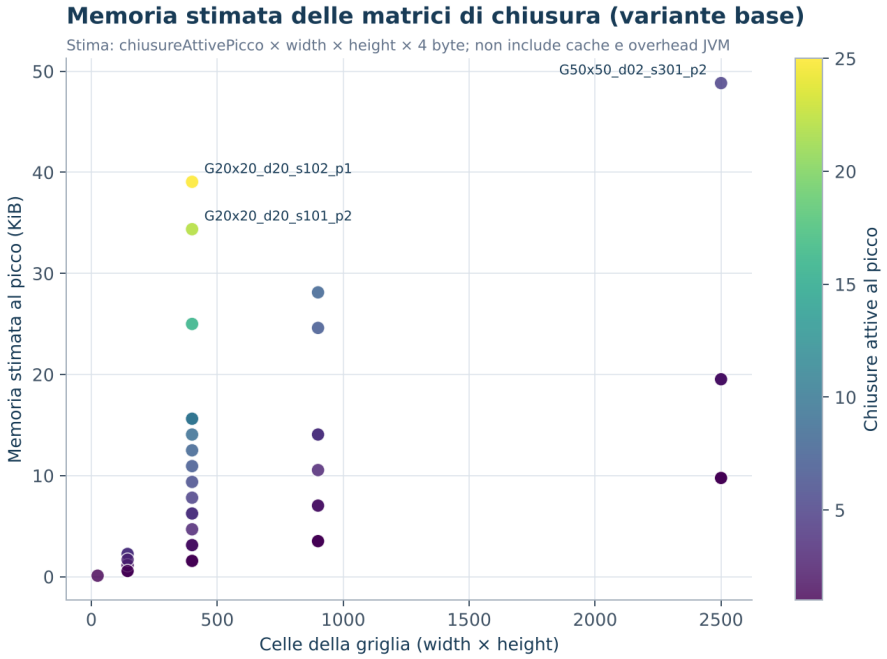


Figura 4: Memoria stimata delle matrici di chiusura attive contemporaneamente, variante `base`, rispetto al numero di celle; il colore indica il numero di matrici.

Memoria. La memoria stimata delle matrici di chiusura (fig. 4) non supera mai 50 000 byte: il massimo corrisponde a cinque matrici della griglia 50×50. Nell'istanza 20×20 più costosa la ricorsione raggiunge profondità 25 con 25 matrici attive insieme, cioè $25 \cdot 400 \cdot 4 = 40\,000$ byte. Come osservato nella sezione 5.1, la stima non comprende la cache.

5.5 Effetto delle ottimizzazioni

La tabella 11 e la fig. 5 confrontano le quattro varianti sui gruppi del piano con ostacoli. Per ogni gruppo sono riportati il numero totale di chiamate ricorsive e la somma dei tempi mediani delle istanze.

	base		senza-cache		senza-ordinamento		limite-propagato	
Gruppo	**chiamate**	**ms**	**chiamate**	**ms**	**chiamate**	**ms**	**chiamate**	**ms**
12×12, 10,4%	93	0,6	97	0,8	339	1,4	88	0,5
20×20, 5%	189	1,6	430	2,6	941	5,6	145	1,4
20×20, 10%	680	3,3	1 008	5,0	8 397	30,1	365	2,0
20×20, 20%	149 515	574,7	608 608	2 542,4	—	—	1 608	6,0
30×30, 5%	1 457	8,5	8 707	66,1	—	—	1 444	8,4
50×50, 2%	588	14,7	588	14,0	—	—	312	10,1

Tabella 11: Chiamate ricorsive totali e somma dei tempi mediani per gruppo e variante; — indica una variante non eseguita nel gruppo.

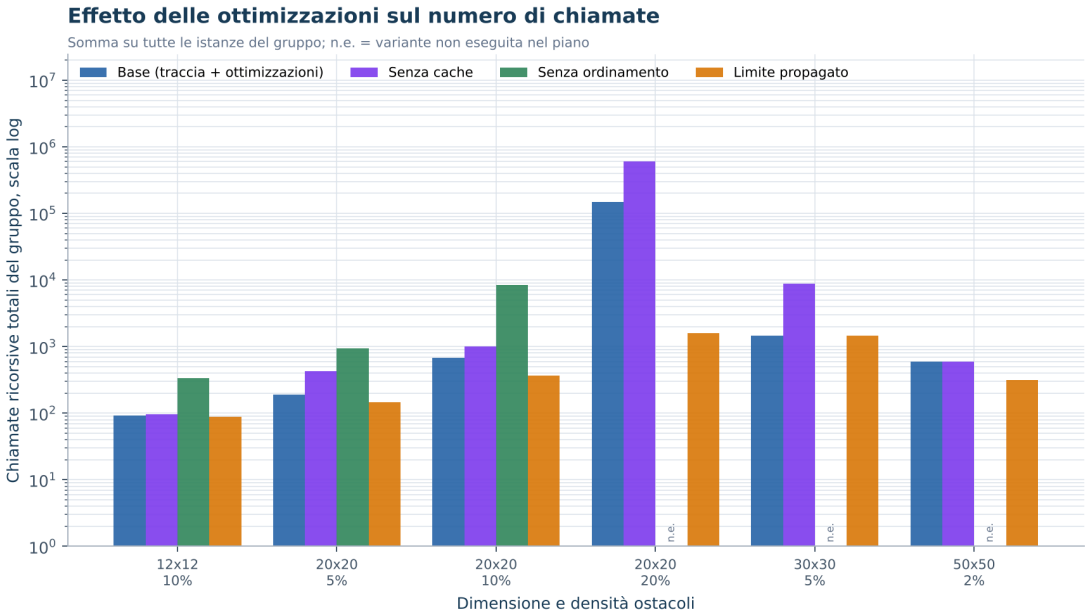


Figura 5: Chiamate ricorsive totali per gruppo e variante, in scala logaritmica.

- **Ordinamento della frontiera.** È l'ottimizzazione più importante fra quelle della versione `base`: senza ordinamento le chiamate aumentano da 3,6 a 12 volte già sulle griglie più facili, e sulle griglie 20×20 al 20% l'esecuzione diventa impraticabile. Visitando prima i candidati con limite inferiore minore, *lunghezzaMin* scende subito e le righe 16 e 17 potano quasi tutti gli altri rami.
- **Cache.** Il guadagno cresce con la difficoltà: senza cache le chiamate sono 4 volte tante a 20×20 al 20% e 6 volte tante a 30×30 al 5%, con tempi 4,4 e 7,8 volte maggiori. Sulle griglie 50×50 al 2% non ci sono sottoproblemi ripetuti e la cache non cambia nulla.
- **Limite propagato.** Sull'intero piano riduce le chiamate da 152 548 a 3 988; nel gruppo 20×20 al 20% le riduce di 93 volte e il tempo di 96 volte. Sui gruppi in cui la versione `base` trova subito il cammino migliore il guadagno è piccolo o nullo, come a 30×30 al 5%.

5.6 Limiti di scala

Il piano principale arriva a 50×50 con il 2% di ostacoli. Per capire dove l'algoritmo smette di essere utilizzabile, la classe `LimitiScala` esegue una serie separata, salvata in `risultati/limiti_scala.csv`, per le varianti `base` e `limite-propagato`:

- griglie quadrate di lato 20, 30, 40, 50, 70 e 100;
- densità di ostacoli 0, 5, 10, 15, 20 e 30%;
- tre seed (1, 2, 3) per combinazione, per un totale di 108 coppie per variante;
- O nella cella libera più vicina al 10% della diagonale e D in quella più vicina al 90%;
- ogni verso eseguito in una JVM separata con limite di **10 s** e memoria massima di **1 GB**; $D \to O$ viene eseguito solo se $O \to D$ è terminato.

Il processo separato permette di registrare un superamento del limite di tempo (`TIMEOUT`) o un `OutOfMemoryError` (`OOM`) e passare alla coppia successiva senza aggiungere a CamminoMin un meccanismo di interruzione, che per questa versione non è richiesto. Ogni verso è una singola esecuzione senza riscaldamento: i tempi comprendono l'avvio della JVM e la compilazione JIT

e non sono confrontabili con quelli del piano. La serie è stata eseguita l'8 novembre 2026 sulla stessa macchina del piano, in circa 18 minuti.

Dimensione	**0%**		**5%**		**10%**		**15%**		**20%**		**30%**	
20×20	3	3	3	3	3	3	3	3	3	3	3	3
30×30	3	3	3	3	2	3	1	3	0	3	0	3
40×40	3	3	3	3	1	3	0	0	0	2	0	2
50×50	3	3	2	2	0	2	0	2	0	1	0	1
70×70	3	3	0	1	0	3	0	0	0	1	0	0
100×100	3	3	1	2	0	0	0	0	0	0	0	0
Totale	**46** su 108 | **73** su 108											

Tabella 12: Coppie completate in entrambi i versi entro i limiti, su 3 per cella: `base` a sinistra, `limite-propagato` a destra.

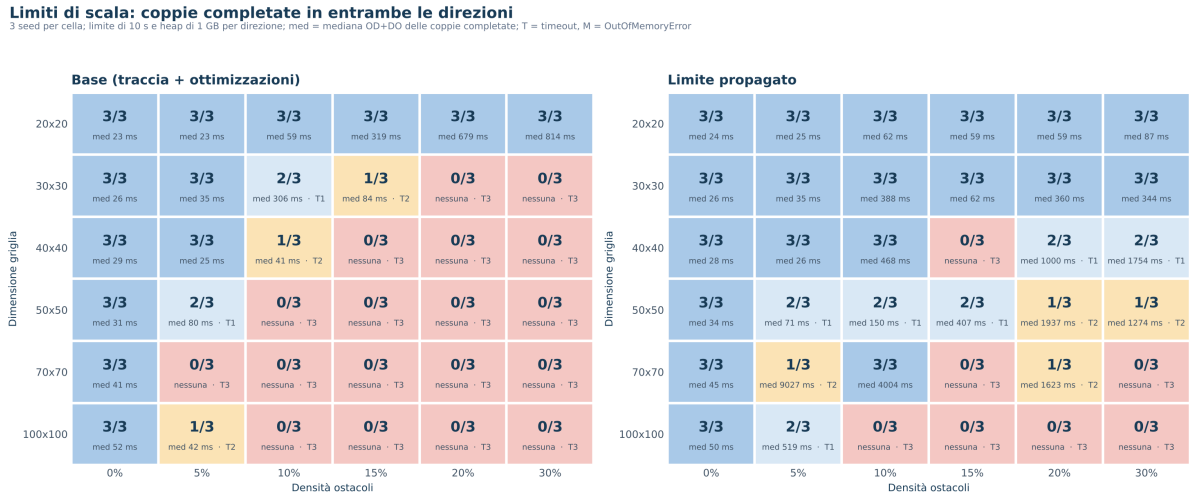


Figura 6: Esito della serie sui limiti di scala per le due varianti. In ogni cella: coppie completate, mediana del tempo $O \to D$ più $D \to O$ delle coppie completate e numero di esecuzioni fermate per tempo (T) o per memoria (M).

Con la variante `base` sono state completate 46 coppie su 108: le altre 62 si sono fermate per il limite di tempo già nel verso $O \to D$. Con il limite propagato le coppie completate sono 73: le stesse 46 più altre 27, mentre 34 si sono fermate nel verso $O \to D$ e una nel verso $D \to O$, sempre per il limite di tempo. Nessuna esecuzione ha esaurito la memoria. Tutte le coppie completate sono simmetriche e, dove entrambe le varianti terminano, le lunghezze coincidono. Numero di chiamate e lunghezze non dipendono dalla macchina; su hardware diverso possono cambiare esito solo le coppie che terminano vicino ai 10 s.

Dalla serie emergono cinque osservazioni.

1. **Senza ostacoli la dimensione non è un problema.** Le 18 coppie allo 0% terminano con una sola chiamata per verso, perché in assenza di ostacoli ogni cella della griglia appartiene al contesto di O. I tempi, fra 23 e 52 ms anche a 100×100, sono quelli di un calcolo di chiusura con la JVM appena avviata.

2. **Per la variante `base` il fattore critico è la densità.** A 20×20 tutte le densità sono gestibili, ma la mediana passa da 23 ms allo 0% a 814 ms al 30%. A 30×30 si perdono

coppie dal 10% in su, a 50×50 resta parzialmente gestibile solo il 5%, a 70×70 e 100×100 nessuna densità diversa da zero è gestibile in modo affidabile.

3. **Il limite propagato sposta il confine.** Completa tutte le coppie 30×30 fino al 30% e più della metà di quelle 40×40 e 50×50 con ostacoli. Sulle 46 coppie completate da entrambe le varianti, le chiamate del verso $O \to D$ scendono da 1 096 774 a 20 140. La coppia più costosa per la variante `base` (20×20, 30%, seed 1) richiede 480 274 chiamate, profondità 24, 2,1 s e fino a 390 MB di heap; con il limite propagato bastano 362 chiamate, 49 ms e 9 MB.

4. **La crescita resta combinatoria.** Anche con il limite propagato le griglie 70×70 e 100×100 superano quasi sempre i 10 s; la coppia completata più costosa (70×70, 5%, seed 2) richiede 508 161 chiamate e 8,8 s nel solo verso $O \to D$. Come previsto dalla sezione 5.1, le ottimizzazioni riducono il numero di chiamate ma non il caso peggiore.

5. **I due versi possono costare in modo molto diverso.** Per ciascuna variante, in 5 coppie completate un verso è più lento dell'altro di oltre 10 volte, per esempio 1 410 ms contro 29 ms a 20×20 con il 15% (seed 3, `base`). Una spiegazione plausibile è che in un verso si trovi presto un cammino corto, che consente di potare molti più rami.

6 Conclusioni

Il progetto realizza i quattro compiti nella versione per una persona: un generatore di griglie riproducibili salvate in JSON, il calcolo della chiusura in una passata di costo lineare, CamminoMin con lunghezza e sequenza dei landmark, e una sperimentazione su 67 istanze più una serie sui limiti di scala. La correttezza è sostenuta dalla simmetria di tutte le esecuzioni, dal confronto con Dijkstra di tutte le varianti e dall'enumerazione completa delle griglie 3×3 per la chiusura.

Il tempo di CamminoMin è spiegato quasi del tutto dal lavoro stimato $C_c \cdot n$ ($R^2 = 0{,}95$), e il numero di chiusure dipende dalla densità e dalla disposizione degli ostacoli più che dalla dimensione: senza ostacoli una griglia 100×100 si risolve in poche decine di millisecondi. L'ordinamento della frontiera è indispensabile e la cache conta sulle istanze difficili. Il limite propagato riduce le chiamate del piano da 152 548 a 3 988 e porta le coppie completate entro 10 s da 46 a 73 su 108, senza cambiare alcuna lunghezza.

6.1 Limitazioni

- **Crescita combinatoria.** Per il numero di chiamate non si ha un limite polinomiale. Con 10 s e 1 GB per verso, `base` è affidabile fino a 20×20 per tutte le densità provate e fino a 40×40 con il 5%; il limite propagato estende la zona a 30×30 (sezione 5.6). Il piano principale resta in questa zona e non rappresenta griglie grandi e dense.
- **Costo per chiusura.** Ogni chiusura scandisce l'intera griglia, anche quando contiene poche celle.
- **Memoria della cache.** La cache conserva ogni sottoproblema risolto anche quando i riusi sono rari: con `base` le chiamate servite dalla cache sono il 13% di quelle del piano.
- **Copertura delle varianti.** `senza-ordinamento` è eseguita solo sulle griglie più facili; sulle altre il suo costo è documentato solo qualitativamente.
- **Misure.** Riscaldamento e cinque ripetizioni senza un framework come JMH [5]; il CSV conserva minimo, mediana e massimo ma non le misure grezze, e le coppie di una stessa griglia non sono osservazioni indipendenti. `memEst` conta solo le matrici di chiusura e il picco di heap della serie è un limite superiore. Le misure sono state eseguite solo su JDK 21.

6.2 Sviluppi futuri

- **Chiusura incrementale.** Limitare il calcolo alla regione raggiunta renderebbe il costo di una chiamata proporzionale alla chiusura invece che alla griglia: è il fattore n del lavoro stimato (eq. (2)).
- **Cache con politica di rimozione**, per ridurre la memoria sulle istanze con pochi riusi.
- **Misure con JMH**, con più ripetizioni e salvataggio delle misure grezze.

7 Utilizzo del codice

7.1 Contenuto della consegna

```
progetto_ASD/
|-- README.md
|-- pom.xml                    build Maven
|-- asd-project.jar            eseguibile precompilato
|-- requirements-report.txt    dipendenze Python per i grafici
|-- src/main/java/mio/         codice applicativo
|-- src/test/java/mio/         test JUnit
|-- risultati/
|   |-- esperimenti.csv        piano sperimentale (67 istanze, 215 righe)
|   |-- riassunti.txt          riassunto di ogni riga del piano
|   |-- limiti_scala.csv       serie sui limiti di scala (108 coppie per variante)
|   |-- griglie_json/          23 griglie del piano
|   |-- grafici/               grafici della relazione
|   `-- genera_grafici.py      script di analisi
`-- relazione/                 sorgenti LaTeX e PDF di questa relazione
```

7.2 Requisiti

- Java 17 o successivo per eseguire il JAR;
- Maven 3.9 o successivo per compilare ed eseguire i test;
- Python 3 con i pacchetti di `requirements-report.txt` per rigenerare i grafici;
- una distribuzione LaTeX con `latexmk` per ricompilare la relazione.

7.3 Esecuzione

Tutti i comandi vanno eseguiti dalla radice della consegna: i file di output sono scritti nella cartella `risultati/`.

Piano sperimentale. Senza argomenti l'applicazione esegue il piano della sezione 5.2, stampa un riassunto per istanza e variante e riscrive `esperimenti.csv`, `riassunti.txt` e le griglie della cartella `griglie_json/`. Richiede meno di un minuto.

```
java -jar asd-project.jar
```

Serie sui limiti di scala. Riscrive `risultati/limiti_scala.csv` per le varianti `base` e `limite-propagato`. Richiede circa 18 minuti, quasi tutti dovuti alle esecuzioni che raggiungono il limite di 10 s; indicando una variante si esegue la serie solo per quella.

```
java -jar asd-project.jar --limiti-scala
java -jar asd-project.jar --limiti-scala limite-propagato
```

Compito Due. Calcola contesto, complemento e frontiera di O e la distanza libera fra O e D su una griglia salvata (sezione 3.5).

```
java -jar asd-project.jar --grid-input
    risultati/griglie_json/scenario_02_12x12_15ost_seed50.json --origin 0,0
    --destination 10,10 --chiusura
```

Compito Tre, istanza singola. Genera una griglia con i parametri indicati, oppure la legge da un file JSON, esegue CamminoMin nei due versi una sola volta con la variante scelta e salva il risultato. Se O o D cadono su un ostacolo viene usata la cella libera più vicina e l'applicazione lo segnala.

```
java -jar asd-project.jar --width 20 --height 20 --obstacles 40 --seed 77 --origin
    0,0 --destination 19,19 --summary true --output risultati/prova.csv
    --grid-output risultati/prova.json
java -jar asd-project.jar --grid-input risultati/prova.json --origin 0,0
    --destination 19,19 --variante limite-propagato
```

Opzione	Significato e valore predefinito
`--width N`, `--height N`	dimensioni della griglia generata (20 e 20)
`--obstacles N`	numero di ostacoli, da 0 a $WH - 1$ (20)
`--seed N`	seed del generatore (42)
`--grid-input f.json`	griglia letta da file invece che generata; esclude le quattro opzioni precedenti e `--grid-output`; nel CSV il seed vale -1
`--origin x,y`	origine O (`0,0`)
`--destination x,y`	destinazione D (angolo opposto all'origine)
`--chiusura`	esegue solo il Compito Due: mappa della chiusura di O, conteggi e dlib(O, D)
`--chiusura-output f.json`	con `--chiusura`, salva gli stati delle celle
`--variante v`	variante di CamminoMin: `base` (predefinita), `senza-cache`, `senza-ordinamento`, `limite-propagato`
`--summary true|false`	stampa del riassunto (`true`)
`--output f.csv`	CSV del risultato (`risultati/esperimento_personalizzato.csv`)
`--grid-output f.json`	griglia generata (`risultati/griglie_json/personalizzata.json`)
`--limiti-scala [v]`	serie sui limiti di scala, per entrambe le varianti principali o solo per v; va usata da sola
`--help`	elenco delle opzioni

Tabella 13: Opzioni della riga di comando.

7.4 Compilazione e test

```
mvn -q clean package
java -jar target/asd-project-1.0-SNAPSHOT.jar --help
```

Il primo comando compila il codice, esegue i 68 test e crea in `target/` un JAR equivalente a `asd-project.jar`; il secondo ne verifica il funzionamento stampando le opzioni, senza riscrivere i risultati. Per eseguire solo i test si usa `mvn -q test`.

7.5 Grafici e relazione

```
python -m pip install -r requirements-report.txt
python risultati/genera_grafici.py
cd relazione
latexmk -pdf Relazione_ASD_2024_25.tex
```

Lo script dei grafici verifica la struttura dei due file CSV e la coerenza delle lunghezze fra varianti prima di disegnare, scrive le immagini in `risultati/grafici/`, da cui la relazione le include direttamente, e stampa i valori riassuntivi riportati nella sezione 5.

A Formato dei file di output

A.1 `esperimenti.csv`

Una riga per istanza e variante, 31 colonne, decimali con il punto. Le sequenze dei landmark sono racchiuse fra virgolette perché contengono virgole; una lunghezza infinita è scritta `INF` e la relativa sequenza `[]`.

Colonne	Significato
`gridId`	identificativo dell'istanza, per esempio `G12x12_d10_s50_p1` (dimensioni, densità in percentuale, seed, numero della coppia)
`variante`	variante di CamminoMin (tabella 6)
`tipoGriglia`	tipo di griglia, sempre `Casuale`
`width`, `height`, `nOstacoli`, `seed`	parametri della griglia
`Ox`, `Oy`, `Dx`, `Dy`	coordinate effettive di O e D
`lunghezzaOD`, `lunghezzaDO`	lunghezze nei due versi, 6 decimali
`sequenzaLandmarkOD`, `sequenzaLandmarkDO`	sequenze dei landmark, ogni elemento nella forma `(x=..,y=..,tipo=..)`
`correttezza`	esito della verifica di simmetria
`frontieraTot`	celle di frontiera di tutte le chiusure calcolate
`falsiRiga16`	volte in cui la riga 16 è risultata falsa
`falsiRiga17`	potature dovute alla sola riga 17
`pruningTotale`	somma delle due precedenti
`chiamateRicorsive`, `cacheHit`	invocazioni totali e invocazioni servite dalla cache
`profonditaMassima`	profondità massima della ricorsione
`chiusureAttivePicco`	matrici di chiusura attive contemporaneamente lungo la pila delle chiamate, al picco
`celleChiusuraTotali`	somma delle dimensioni delle chiusure calcolate
`memoriaMatriciPiccoBytesStimata`	`chiusureAttivePicco` $\cdot\, W \cdot H \cdot 4$ byte
`warmupEseguiti`, `ripetizioniTempo`	riscaldamenti e ripetizioni misurate
`tempoMinNs`, `tempoMedianoNs`, `tempoMaxNs`	tempi delle ripetizioni in nanosecondi, versi $O \to D$ e $D \to O$ insieme

Tabella 14: Colonne di `esperimenti.csv`. Tutti i contatori sommano i due versi.

A.2 `limiti_scala.csv`

Una riga per variante e coppia, 24 colonne.

Colonne	Significato
`variante`	`base` oppure `limite-propagato`
`width`, `height`, `densitaPerc`, `nOstacoli`, `seed`	parametri della griglia
`Ox`, `Oy`, `Dx`, `Dy`	coordinate di O e D
`esitoOD`, `esitoDO`	`OK`, `TIMEOUT`, `OOM`, `STACK` o `ERRORE`; `esitoDO` vale `NON_ESEGUITO` se il verso $O \to D$ non è terminato
`lunghezzaOD`, `lunghezzaDO`	lunghezze, `INF` o vuote se il verso non è terminato
`simmetria`	`true` o `false` se entrambi i versi sono terminati, altrimenti `ND`
`tempoODms`, `tempoDOms`	tempo di una singola esecuzione a freddo, in millisecondi
`chiamateOD`, `profonditaOD`, `frontieraOD`, `cacheHitOD`	contatori del verso $O \to D$
`piccoHeapMBOD`	somma dei picchi delle aree di heap della JVM, in MB
`timeoutMs`, `heapMax`	limiti applicati a ogni verso

Tabella 15: Colonne di `limiti_scala.csv`.

A.3 `riassunti.txt`

Una riga per ogni riga di `esperimenti.csv`, nel formato del riassunto della sezione 4.7, nello stesso ordine del CSV.

A.4 Griglie JSON

Ogni file di `risultati/griglie_json/` contiene un array di righe; ogni elemento è lo stato della cella secondo la tabella 2 (0 libera, 128 ostacolo). Il nome del file riporta numero dello scenario, dimensioni, ostacoli e seed, per esempio `scenario_02_12x12_15ost_seed50.json`.

Riferimenti bibliografici

[1] M. Zanella, *Progetto di Algoritmi e Strutture Dati, A.A. 2024/25*, testo dell'elaborato, Università degli Studi di Brescia.

[2] E. W. Dijkstra, *A note on two problems in connexion with graphs*, Numerische Mathematik, 1:269–271, 1959.

[3] Oracle, *Java SE 17 API: `java.util.Random`*, `https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Random.html`.

[4] Oracle, *Java SE 17 API: `java.util.BitSet`*, `https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/BitSet.html`.

[5] OpenJDK, *Java Microbenchmark Harness (JMH)*, `https://github.com/openjdk/jmh`.